

Dynamic Programming

DP technique

Guideline

W activity
selection

0-1 Knapsack

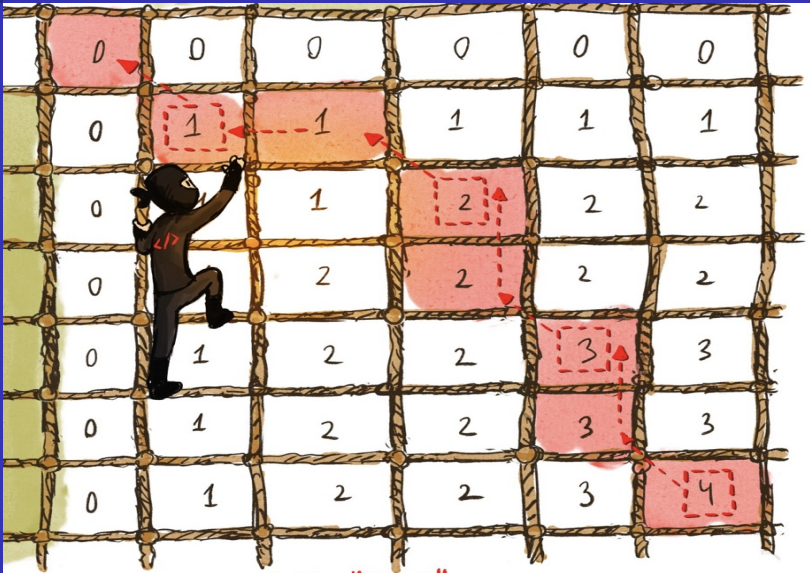
DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring



Dynamic Programming

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

For a gentle introduction to DP see Chapter 6 in DPV, KT and CLRS also have a chapter devoted to DP.

Richard Bellman: *An introduction to the theory of dynamic programming* RAND, 1953



Dynamic programming is a powerful technique for efficiently implement *recursive algorithms* by storing partial results and re-using them when needed.

Dynamic Programming

Dynamic Programming works efficiently when:

- **Subproblems:** There must be a way of breaking the global optimization problem into subproblems, each having a similar structure to the original problem but smaller size.
- **Optimal sub-structure:** An optimal solution to a problem must be a composition of optimal subproblem solutions, using a relatively simple combining operation.
- **Repeated subproblems:** The recursive algorithm solves a small number of distinct subproblems, but they are repeatedly solved many times.

This last property allows us to take advantage of **memoization**, store intermediate values, using the appropriate dictionary data structure, and reuse when needed.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Difference with greedy

DP technique

Guideline

W activity selection

0-1 Knapsack

DP for pairing sequences

Framework

Edit distance

Longest common subsequence (LCS)

Longest common substring

- Greedy problems have the **greedy choice property**: locally optimal choices lead to globally optimal solution. **We solve recursively one subproblem**
- I.e. In DP we solve all possible subproblems, while in greedy we are bound for the initial choice

Difference with divide and conquer

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- Both require recursive programming with subproblems with a similar structure to the original
- D & C breaks a problems into a small number of subproblems each of them with size a fraction of the original size (size/b).
- In DP, we break into many subproblems **with smaller size**, but often, their sizes are not a fraction of the initial size.

Guideline to implement Dynamic Programming

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

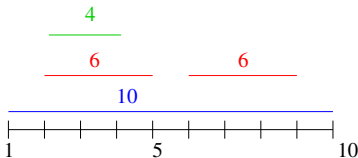
Longest common
substring

- 1 *Characterize the structure of subproblems:* make sure space of subproblems is not exponential. Define variables.
- 2 Define recursively the value of an optimal solution: **Find the correct recurrence**, with solution to larger problem as a function of solutions of sub-problems.
- 3 *Compute, memoization/bottom-up, the cost of a solution:* using the recursive formula, tabulate solutions to smaller problems, until arriving to the value for the whole problem.
- 4 *Construct an optimal solution:* compute additional information to **trace-back** an optimal solution from optimal value.

WEIGHTED ACTIVITY SELECTION problem

WEIGHTED ACTIVITY SELECTION problem: Given a set $S = \{1, 2, \dots, n\}$ of activities to be processed by a single resource. Each activity i has a start time s_i and a finish time f_i , with $f_i > s_i$, and a weight w_i . Find the set of mutually compatible activities such that it maximizes $\sum_{i \in S} w_i$

Recall: We saw that some greedy strategies did not provide always a solution to this problem.



DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

W Activity Selection: looking for a recursive solution

DP technique

Guideline

W activity selection

0-1 Knapsack

DP for pairing sequences

Framework

Edit distance

Longest common subsequence (LCS)

Longest common substring

- Let us think of a backtracking algorithm for the problem.
- The solution is a selection of activities, i.e., a subset $S \subseteq \{1, \dots, n\}$.
- We can adapt the backtracking algorithm to compute all subsets.
- When processing element i , we branch
 - i is in the solution S , then all activities that overlap with i cannot be in S .
 - i is not in S .

W Activity Selection: looking for a recursive solution

Each backtracking call receives a partial solution (S) and a candidate set (C), those activities that are compatible with the ones in S . It returns the weight of the best solution enlarging S .

WAS-1 (S, C)

if $C = \emptyset$ **then**

return ($W(S)$)

Let i be an element in C ; $C = C - \{i\}$;

Let A be the set of activities in C that overlap with i

return ($\max\{\mathbf{WAS-1}(S \cup \{i\}, C - A), \mathbf{WAS-1}(S, C)\}$)

The recursion tree have branching 2 and height $\leq n$, so size is $O(2^n)$.

How many subproblems appear here? hard to count better than $O(2^n)$.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

W Activity Selection: looking for a recursive solution

For the unweighted case, the greedy algorithm made use of a particular ordering that helped to discard overlapping tasks.

Assume that the activities are sorted by finish time, i.e.,

$$f_1 \leq f_2 \leq \dots \leq f_n.$$

- We can look to the incompatible activities that appear before activity i (finishing before t_i) when dealing with activity i .
- An incompatibility with a task j with $t_j \geq t_i$ will be discovered when dealing with task j .
- Note that activity i is not compatible with activity $j < i$ when $t_j \geq s_i$.
- Then i can be incompatible with a **contiguous** set of activities $i - 1, i - 2, \dots, j$.

DP technique

Guideline

W activity selection

0-1 Knapsack

DP for pairing sequences

Framework

Edit distance

Longest common subsequence (LCS)

Longest common substring

W Activity Selection: looking for a recursive solution

This suggest the following backtracking algorithm.

WAS-2 (S, i)

if $i == 1$ then

 return ($W(S) + w_1$)

if $i == 0$ then

 return ($W(S)$)

Let j be the largest integer $j < i$ such that $f_j \leq s_i$, 0 if none is compatible.

return ($\max\{\mathbf{WAS-2}(S \cup \{i\}, j), \mathbf{WAS-2}(S, i - 1)\}$)

WAS-2 ($\emptyset, n$) will return the cost of an optimal solution, as we are considering adding or not i to the solution and discarding all incompatible tasks when choosing i .

The algorithm has cost $O(2^n)$.

Inside the code, we are not using S , only $W(S)$.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

W Activity Selection: looking for a recursive solution

DP technique

Guideline

W activity selection

0-1 Knapsack

DP for pairing sequences

Framework

Edit distance

Longest common subsequence (LCS)

Longest common substring

This suggests the following backtracking algorithm that receives $W(S)$ instead of S .

WAS-3 (W, i)

if $i == 1$ then

return ($W + w_1$)

if $i == 0$ then

return (W)

Let j be the largest integer $j < i$ such that $f_j \leq s_i$, 0 if none is compatible.

return ($\max\{\mathbf{WAS-2}(W + w_i, j), \mathbf{WAS-2}(W, i - 1)\}$)

WAS-3 ($\emptyset, n$) will return the cost of an optimal solution. Still, the algorithm has cost $O(2^n)$.

DP from WAS-3: a recurrence

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- We have n activities with $f_1 \leq f_2 \leq \dots \leq f_n$ and weights w_i , $1 \leq i \leq n$.
- **Supproblems** calls $\text{WAS-3}(W, i)$
 - i defines the subproblem: Maximize the weight of the activity selection for activities $\{1, \dots, i\}$, for $0 \leq i \leq n$.
 - A recursive call returns this maximum.
 - There are only $O(n)$ subproblems!
- Let $\text{Opt}(j)$ be the value of an optimal solution O_j to the sub problem consisting of activities in the range 1 to j .

DP from WAS-3: a recurrence

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

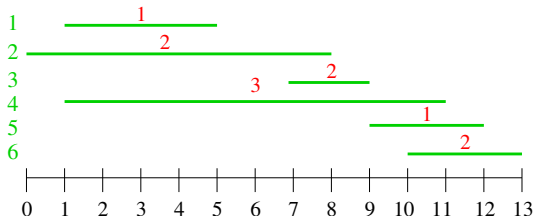
Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Define $p(i)$ to be the largest integer $j < i$ such that i and j are disjoint ($p(i) = 0$ if no disjoint $j < i$ exists).



$p(1)=0$

$p(2)=0$

$p(3)=1$

$p(4)=0$

$p(5)=3$

$p(6)=3$

DP from WAS-3: a recurrence

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Reinterpreting WAS-3 bottom-up, using $p(i)$, we get

$$\text{Opt}(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\{(\text{Opt}(p[j]) + w_j), \text{Opt}[j - 1]\} & \text{if } j \geq 1 \end{cases}$$

We add activity 0 for compatibility with p .

DP from WAS-2: a recurrence

$$\text{Opt}(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\{(\text{Opt}(p[j]) + w_j), \text{Opt}[j - 1]\} & \text{if } j \geq 1 \end{cases}$$

Correctness: The base case is correct. From the previous discussion, we have two cases:

1.- $j \in O_j$:

- As j is part of the solution, no jobs $\{p(j) + 1, \dots, j - 1\}$ are in O_j ,
- $O_j - \{j\}$ must be an optimal solution for $\{1, \dots, p[j]\}$, otherwise then $O'_j = O_{p[j]} \cup \{j\}$ will be better (**optimal substructure**)

2.- If $j \notin O_j$: then O_j is an optimal solution to $\{1, \dots, j - 1\}$.

DP from WAS-3: Computing p values

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

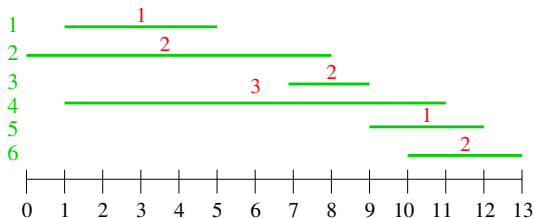
Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- We need to compute efficiently $p(i)$
 - sort the activities by increasing values of start time.
 - merge the sorted list of finishing times and the sorted list of start times, in case of tie put before the finish times.
 - $p[j]$ is the last activity whose finish time precedes s_j in the combined order, activity 0, if no finish time precedes s_j
- We can thus compute the p values in $O(n \lg n + n) = O(n \lg n)$ time.

DP from WAS-3: computing p values



$p(1)=0$
 $p(2)=0$
 $p(3)=1$
 $p(4)=0$
 $p(5)=3$
 $p(6)=3$

Sorted finish times: 1:5, 2:8, 3:9, 4:11, 5:12, 6:13

Sorted start times: 2:0, 1:1, 4:1, 3:7, 5:9, 6:10

Merged sequence: 2:0, 1:1, 4:1, 1:5, 3:7, 3:9, 5:9, 6:10, 5:12, 6:13

DP from WAS-3: Preprocessing

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Considering the set of activities S , we start by a pre-processing phase:

- Sort the activities by increasing values of finish times.
- Compute the values of $p[i]$,
- This can be done in $O(n \lg n)$

DP from WAS-2: Memoization

Assuming that tasks are sorted and all $p(j)$ are computed and tabulated in $P[1 \dots n]$

We keep a table $W[n+1]$, at the end $W[i]$ will hold the weight of an optimal solution for subproblem $\{1, \dots, i\}$. Initially, set all entries to -1 and $W[0] = 0$.

```
R-Opt ( $j$ )  
  if  $W[j] \neq -1$  then  
    return ( $W[j]$ )  
  else  
     $W[j] = \max(w_j + \mathbf{R-Opt}(P[j]), \mathbf{R-Opt}(j-1))$   
    return  $W[j]$ 
```

No subproblem is solved more than once, so cost is $O(n \log + n) = O(n \log n)$

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

DP from WAS-3: Iterative

We assume that tasks are sorted and all $p(j)$ are computed and tabulated in $P[1 \dots n]$

We keep a table $W[n+1]$, at the end $W[i]$ will hold the weight of an optimal solution for subproblem $\{1, \dots, i\}$.

Opt-Val (n)

$W[0] = 0$

for $j = 1$ to n **do**

$W[j] = \max(W[P[j]] + w_j, W[j-1])$

return $W[n]$

Time complexity: $O(n \lg n + n)$.

Notice: Both algorithms gave only the numerical max. weight

We have to keep more info to recover a solution from $W[n]$.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

DP from WAS-2: Returning an optimal solution

To get also the list of activities in an optimal solution, **we use W to recover the decision taken in computing $W[n]$.**

```
Find-Opt ( $j$ )  
if  $j = 0$  then  
    return  $\emptyset$   
else if  $W[p[j]] + w_j > W[j - 1]$  then  
    return  $\{j\} \cup \text{Find-Opt}(p[j])$   
else  
    return  $(\text{Find-Opt}(j - 1))$ 
```

Time complexity: $O(n)$

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

DP for Weighted Activity Selection

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- We started from a suitable recursive algorithm, which runs $O(2^n)$ but solves only $O(n)$ different subproblems.
- Perform some preprocessing.
- Compute the weight of an optimal solution to each of the $O(n)$ subproblems.
- Guided by optimal value, obtain an optimal solution .

0-1 KNAPSACK

(This example is from Section 6.4 in Dasgupta, Papadimitriou, Vazirani's book.)

0-1 KNAPSACK: Given as input a set of n items that can NOT be fractioned, item i has weight w_i and value v_i , and a maximum permissible weight W .

QUESTION: select a set of items S that maximize the profit.

Recall that we can **NOT** take fractions of items.



DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Subproblems and recurrence

Input: $(w_1, \dots, w_n), (v_1, \dots, v_n), W$.

- Let $S \subseteq \{1, \dots, n\}$ be an optimal solution to the problem
The optimal benefit is $\sum_{i \in S} v_i$
- With respect to the last item we have two cases:
 - $n \notin S$, then S is an optimal solution to the problem
 $(w_1, \dots, w_{n-1}), (v_1, \dots, v_{n-1}), W$
 - $n \in S$, then $S - \{n\}$ is an optimal solution to the problem
 $(w_1, \dots, w_{n-1}), (v_1, \dots, v_{n-1}), W - w_n$
- in both cases we get an optimal solution of a subproblem in which the last item is removed and in which the maximum weight can be W or a value smaller than W .
- This identifies subproblems of the form $[i, x]$ that are knapsack instances in which the set of items is $\{1, \dots, i\}$ and the maximum weight that can hold the knapsack is x .

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Subproblems and recurrence

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Let $v[i, x]$ be the maximum value (optimum) we can get from objects $\{1, 2, \dots, i\}$ within total weight $\leq x$.

To compute $v[i, x]$, the two possibilities we have considered give rise to the recurrence:

$$v[i, x] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \max v[i - 1, x - w_i] + v_i, v[i - 1, x] & \text{otherwise} \end{cases}$$

DP algorithm: tabulating

Define a table $P[n + 1, W + 1]$ to hold optimal values for the corresponding subproblem.

```
Knapsack( $i, x$ )  
for  $i = 0$  to  $n$  do  
     $P[i, 0] = 0$   
for  $x = 1$  to  $W$  do  
     $P[0, x] = 0$   
for  $i = 1$  to  $n$  do  
    for  $x = 1$  to  $W$  do  
         $P[i, x] = \max\{P[i - 1, x], P[i - 1, x - w[i]] + v[i]\}$   
return  $P[n, W]$ 
```

The number of steps is $O(nW)$ which is **pseudopolynomial**.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

An example

i	1	2	3	4	5
w_i	1	2	5	6	7
v_i	1	6	18	22	28

$W = 11.$

		w											
		0	1	2	3	4	5	6	7	8	9	10	11
/	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	1	1	1	1	1	1	1	1	1	1	1
	2	0	1	6	7	7	7	7	7	7	7	7	7
	3	0	1	6	7	7	18	19	24	25	25	25	25
	4	0	1	6	7	7	18	22	23	28	29	29	40
	5	0	1	6	7	7	18	22	28	29	34	35	40

For instance, $v[4, 10] = \max\{v[3, 10], v[3, 10 - 6] + 22\} = \max\{25, 7 + 22\} = 29.$

$v[5, 11] = \max\{v[4, 11], v[4, 11 - 7] + 28\} = \max\{40, 4 + 28\} = 40.$

DP technique

Guideline

W activity selection

0-1 Knapsack

DP for pairing sequences

Framework

Edit distance

Longest common subsequence (LCS)

Longest common substring

Recovering the solution

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

To compute the actual subset $S \subseteq I$ that is the solution, we modify the algorithm to compute also a Boolean table $K[n+1, W+1]$, so that $K[i, x]$ is 1 when the max is attained in the second alternative ($i \in S$), 0 otherwise.

```
Knapsack( $i, x$ )  
for  $i = 0$  to  $n$  do  
     $P[i, 0] = 0$ ;  $K[i, 0] = 0$   
for  $x = 1$  to  $W$  do  
     $P[0, x] = 0$ ;  $K[0, x] = 0$   
for  $i = 1$  to  $n$  do  
    for  $x = 1$  to  $W$  do  
        if  $P[i-1, x] \geq$   
            $P[i-1, x - w[i]] + v[i]$  then  
             $P[i, x] = P[i-1, x]$ ;  
             $K[i, x] = 0$   
        else  
             $P[i, x] =$   
                $P[i-1, x - w[i]] + v[i]$ ;  
             $K[i, x] = 1$   
    return  $P[n, W]$ 
```

Complexity: $O(nW)$

An example

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

	0	1	2	3	4	5	6	7	8	9	10	11
0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0
1	0 0	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1
2	0 0	1 0	6 1	7 1	7 1	7 1	7 1	7 1	7 1	7 1	7 1	7 1
3	0 0	1 0	6 0	7 0	7 0	18 1	19 1	24 1	25 1	25 1	25 1	25 1
4	0 0	1 0	6 0	7 0	7 0	18 1	22 1	23 1	28 1	29 1	29 1	40 1
5	0 0	1 0	6 0	7 0	7 0	18 0	22 0	28 1	29 1	34 1	35 1	40 0

Recovering the solution

- To compute an optimal solution $S \subseteq I$, we use K to trace backwards the elements in the solution.
- $K[i, x]$ is 1 when the max is attained in the second alternative: $i \in S$.

```
x = W, S = ∅  
for i = n downto 1 do  
  if K[i, x] = 1 then  
    S = S ∪ {i}  
    x = x - wi  
Output S
```

Complexity: $O(nW)$

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

An example

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

i	1	2	3	4	5
w_i	1	2	5	6	7
v_i	1	6	18	22	28

$$W = 11.$$

	0	1	2	3	4	5	6	7	8	9	10	11
0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0	0 0
1	0 0	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1	1 1
2	0 0	1 0	6 1	7 1	7 1	7 1	7 1	7 1	7 1	7 1	7 1	7 1
3	0 0	1 0	6 0	7 0	7 0	18 1	19 1	24 1	25 1	25 1	25 1	25 1
4	0 0	1 0	6 0	7 0	7 0	18 1	22 1	23 1	28 1	29 1	29 1	40 1
5	0 0	1 0	6 0	7 0	7 0	18 0	22 0	28 1	29 1	34 1	35 1	40 0

$K[5, 11] \rightarrow K[4, 11] \rightarrow K[3, 5] \rightarrow K[2, 0]$. So $S = \{4, 3\}$

Complexity

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

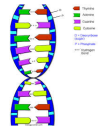
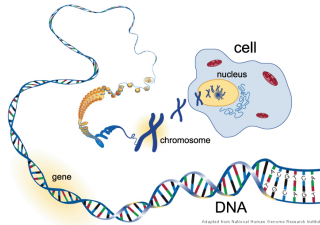
Longest common
subsequence (LCS)

Longest common
substring

The 0-1 KNAPSACK is NP-complete.

- 0-1 KNAPSACK, has complexity $O(nW)$, and its length is $O(n \lg M)$ taking $M = \max\{W, \max_i w_i, \max_i v_i\}$.
- If W requires k bits, the cost and space of the algorithm is $n2^k$, exponential in the length W . However the DP algorithm works fine when $W = \Theta(n)$, here $k = O(\log n)$.
- Consider the **unary knapsack problem**, where all integers are coded in unary ($7=111111$). In this case, the complexity of the DP algorithm is polynomial on the size, i.e., **UNARY KNAPSACK** $\in P$.

Matching DNA sequences



- DNA, is the hereditary material in almost all living organisms. They can reproduce by themselves.
- Its function is like a program unique to each individual organism that rules the working and evolution of the organism.
- Model as a string of 3×10^9 characters over $\{A, T, G, C\}$.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Computational genomics: Some questions

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- When a new gene is discovered, one way to gain insight into its working, is to find well known genes (not necessarily in the same species) which match it closely. Biologists suggest a generalization of edit distance as a definition of approximately match.
- GenBank (<https://www.ncbi.nlm.nih.gov/genbank/>) has a collection of $> 10^{10}$ well studied genes, BLAST is a software to do fast searching for similarities between a gene and those in a DB of genes.
- **Sequencing DNA:** consists in the determination of the order of DNA bases, in a short sequence of 500-700 characters of DNA. To get the global picture of the whole DNA chain, we generate a large amount of DNA sequences and try to assemble them into a coherent DNA sequence. This last part is usually a difficult one, as the position of each sequence in the global DNA chain is not known beforehand.

Evolution DNA

T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---

Mutation

T	A	C	A	C	T	A	C	G
---	---	---	---	---	---	---	---	---

Delete

T	X	C	A	G	X	A	C	G
---	--------------	---	---	---	--------------	---	---	---

T	C	A	G	A	C	G
---	---	---	---	---	---	---

Insertion

A	T	C	A	G	A	C	G
---	---	---	---	---	---	---	---

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

How to compare sequences?

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

A	C	C	G	G	T	C	G	A	G	T
---	---	---	---	---	---	---	---	---	---	---

 ...

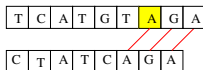
?

G	T	C	G	T	T	C	G	G	A	A
---	---	---	---	---	---	---	---	---	---	---

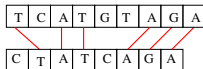
 ...

Three problems

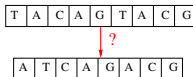
Longest common substring: Substring = consecutive characters in the string.



Longest common subsequence: Subsequence = ordered chain of characters (might have gaps).



Edit distance: Convert one string into another one using a given set of operations.



DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

The EDIT DISTANCE problem

(Section 6.3 in Dasgupta, Papadimitriou, Vazirani's book.)

Information

edit dist = 4

Diagram illustrating the edit distance between the strings "Information" and "f a r a m t o i n". The diagram shows the alignment of the two strings with edit operations indicated by red lines and labels:

- replace: 'I' to 'f'
- delete: 'n' (in "Information")
- insert: 'a' (in "f a r a m t o i n")
- transpose: 'r' and 'a' (in "f a r a m t o i n")

The **edit distance** between strings $X = x_1 \cdots x_n$ and $Y = y_1 \cdots y_m$ is defined to be the **minimum** number of *edit operations* needed to transform X into Y .

All the operations are done on X

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Edit distance: Applications

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- Computational genomics: **evolution between generations**, i.e. between strings on $\{A, T, G, C, -\}$.
- Natural Language Processing: distance, between strings on the alphabet.
- Text processor, suggested corrections

EDIT DISTANCE: Levenshtein distance

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

In the Levenshtein distance the set of operations are

- $\text{insert}(X, i, a) = x_1 \cdots x_i a x_{i+1} \cdots x_n.$
- $\text{delete}(X, i) = x_1 \cdots x_{i-1} x_{i+1} \cdots x_n$
- $\text{modify}(X, i, a) = x_1 \cdots x_{i-1} a x_{i+1} \cdots x_n.$

the cost of modify is 2, and the cost of insert/delete is 1.

To simplify, in the following we assume that *the cost of each operation is 1*.

For other operations and costs the structure of the DP will be similar.

Exemple-1

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

$X = aabab$ and $Y = babb$
 $aabab = X$

$X' = \text{insert}(X, 0, b)$ $b aabab$

$X'' = \text{delete}(X', 2)$ $babab$

$X'' = \text{delete}(X'', 4)$ $babb$

$X = aabab \rightarrow Y = babb$

A shortest edit distance

$aabab = X$

$X' = \text{modify}(X, 1, b)$ $babab$

$Y = \text{delete}(X', 4)$ $babb$

Use dynamic programming.

The structure of an optimal solution

- In a solution O with minimum edit distance from $X = x_1 \cdots x_n$ to $Y = y_1 \cdots y_m$, we have three possible alignments for the last terms

(1)	(2)	(3)
x_n	—	x_n
—	y_m	y_m

- In (1), O performs **delete** x_n , and it transforms optimally, $x_1 \cdots x_{n-1}$ into $y_1 \cdots y_m$.
- In (2), O performs **insert** y_m at the end of x , and it transforms optimally, $x_1 \cdots x_n$ into $y_1 \cdots y_{m-1}$.
- In (3), if $x_n \neq y_m$, O performs **modify** x_n by y_m , otherwise O , aligns them without cost. Furthermore O transforms optimally $x_1 \cdots x_{n-1}$ into $y_1 \cdots y_{m-1}$.

The recurrence

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Let $X[i] = x_1 \cdots x_i$, $Y[j] = y_1 \cdots y_j$.

$E[i, j]$ = edit distance from $X[i]$ to $Y[j]$ is the maximum of

- **I** put y_j at the end of x : $E[i, j - 1] + 1$
- **D** delete x_i : $E[i - 1, j] + 1$
- if $x_i \neq y_j$, **M** change x_i into y_j : $E[i - 1, j - 1] + 1$,
otherwise $E[i - 1, j - 1]$

Edit distance: Recurrence

Adding the base cases, we have the recurrence

$$E[i, j] = \begin{cases} j & \text{if } i = 0 \text{ (converting } \lambda \rightarrow Y[j]) \\ i & \text{if } j = 0 \text{ (converting } X[i] \rightarrow \lambda) \\ \min \begin{cases} E[i-1, j] + 1 & \text{if D} \\ E[i, j-1] + 1, & \text{if I} \\ E[i-1, j-1] + \delta(x_i, y_j) & \text{otherwise} \end{cases} & \text{otherwise} \end{cases}$$

where

$$\delta(x_i, y_j) = \begin{cases} 0 & \text{if } x_i = y_j \\ 1 & \text{otherwise} \end{cases}$$

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Computing the optimal costs and pointers

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

```
Edit(X, Y)
for i = 0 to n do
    E[i, 0] = i
for j = 0 to m do
    E[0, j] = j
for i = 1 to n do
    for j = 1 to m do
         $\delta = 0$ 
        if  $x_i \neq y_j$  then
             $\delta = 1$ 
         $E[i, j] = E[i, j - 1] + 1$   $b[i, j] = \uparrow$ 
        if  $E[i - 1, j - 1] + \delta < E[i, j]$  then
             $E[i, j] = E[i - 1, j - 1] + \delta$ ,  $b[i, j] := \nwarrow$ 
        if  $E[i - 1, j] + 1 < E[i, j]$  then
             $E[i, j] = E[i - 1, j] + 1$ ,  $b[i, j] := \leftarrow$ 
```

Space and time
complexity:

$O(nm)$.

$\leftarrow$ is a **I**
operation,
 $\uparrow$ is a **D**
operation, and
 $\nwarrow$ is either a **M**
or a
no-operation.

Computing the optimal costs: Example

$X = \text{aabab}$; $Y = \text{babb}$. Therefore, $n = 5$, $m = 4$

		0	1	2	3	4
		λ	b	a	b	b
0	λ	0	$\leftarrow 1$	$\leftarrow 2$	$\leftarrow 3$	$\leftarrow 4$
1	a	$\uparrow 1$	$\swarrow 1$	$\swarrow 1$	$\leftarrow 2$	$\leftarrow 3$
2	a	$\uparrow 2$	$\swarrow 2$	$\swarrow 1$	$\leftarrow 2$	$\leftarrow 3$
3	b	$\uparrow 3$	$\swarrow 2$	$\uparrow 2$	$\swarrow 1$	$\swarrow 2$
4	a	$\uparrow 4$	$\uparrow 3$	$\swarrow 2$	$\uparrow 2$	$\swarrow 2$
5	b	$\uparrow 5$	$\swarrow 4$	$\uparrow 3$	$\uparrow 2$	$\swarrow 2$

$\leftarrow$ is a **I** operation, $\uparrow$ is a **D** operation, and
 $\swarrow$ is either a **M** or a **no-operation**.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Obtain Y in edit distance from X

Uses as input the arrays E and b .

The first call to the algorithm is **con-Edit** (n, m)

```
con-Edit( $i, j$ )  
  if  $i = 0$  or  $j = 0$  then  
    return  
  if  $b[i, j] = \nwarrow$  and  $x_i = y_j$  then  
    change( $X, i, y_j$ ); con-Edit( $i - 1, j - 1$ )  
  if  $b[i, j] = \uparrow$  then  
    delete( $X, i$ ); con-Edit( $i - 1, j$ )  
  if  $b[i, j] = \leftarrow$  then  
    insert( $X, i, y_j$ ), con-Edit( $i, j - 1$ )
```

This algorithm has time complexity $O(nm)$.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

The Longest Common Subsequence

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

(Section 15.4 in CormenLRS' book.)

- $Z = z_1 \cdots z_k$ is a **subsequence** of X if there is a subsequence of integers $1 \leq i_1 < i_2 < \dots < i_k \leq n$ such that $z_j = x_{i_j}$.

TTT is a subsequence of $ATATAT$.

- If Z is a subsequence of X and Y , then Z is a **common subsequence** of X and Y .

LCS Given sequences $X = x_1 \cdots x_n$ and $Y = y_1 \cdots y_m$, compute the longest common subsequence Z .

DP approach: Characterization of optimal solution

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Let $X = x_1 \cdots x_n$ and $Y = y_1 \cdots y_m$ and let Z be a longest common subsequence (lcs). Then,

- $Z = x_{i_1} \cdots x_{i_k} = y_{j_1} \cdots y_{j_k}$
- There are no i, j , with $i > i_k$ and $j > j_k$, s.t. $x_i = y_j$. Otherwise, Z will not be optimal.
- $a = x_{i_k}$ might appear after i_k in X , but not after j_k in Y , or viceversa.
- There is an optimal solution in which i_k and j_k are the last occurrence of a in X and Y respectively.

DP approach: Characterization of optimal solution

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Let $X = x_1 \cdots x_n$ and $Y = y_1 \cdots y_m$ and let $Z = x_{i_1} \cdots x_{i_k} = y_{j_1} \cdots y_{j_k}$ a lcs s.t. the index of the final common symbol in Z is its last occurrence in both X and Y .

Let $X^- = x_1 \cdots x_{n-1}$ and $Y^- = y_1 \cdots y_{m-1}$

- Let us look at x_n and y_m .
- If $x_n = y_m$, $i_k = n$ and $j_k = m$ so, $x_{i_1} \cdots x_{i_{k-1}}$ is a lcs of X^- and Y^- .

DP approach: Characterization of optimal solution

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Let $X = x_1 \cdots x_n$ and $Y = y_1 \cdots y_m$ and let $Z = x_{i_1} \cdots x_{i_k} = y_{j_1} \cdots y_{j_k}$ a lcs s.t. the index of the final common symbol in Z is its last occurrence in X and Y .

Let $X^- = x_1 \cdots x_{n-1}$ and $Y^- = y_1 \cdots y_{m-1}$

- Let us look at x_n and y_m .
- If $x_n \neq y_m$,
 - If $i_k < n$ and $j_k < m$, Z is a lcs of X^- and Y^- .
 - If $i_k = n$ and $j_k < m$, Z is a lcs of X and Y^- .
 - If $i_k < n$ and $j_k = m$, Z is a lcs of X^- and Y .
 - The last two include the first one!

DP approach: Subproblems

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Subproblems = lcs of pairs of prefixes of the initial strings.

Notation:

- $X[i] = x_1 \dots x_i$, for $0 \leq i \leq n$
- $Y[j] = y_1 \dots y_j$, for $0 \leq j \leq m$
- $c[i, j]$ = length of the LCS of $X[i]$ and $Y[j]$.
- Want $c[n, m]$ i.e. length of the LCS for X and Y .

DP approach: Recursion

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Therefore, given X and Y

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i - 1, j - 1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j - 1], c[i - 1, j]) & \text{otherwise} \end{cases}$$

The recursive algorithm

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

```
LCS( $X, Y$ )  
   $n = X.size(); m = Y.size()$   
  if  $n = 0$  or  $m = 0$  then  
    return 0  
  else if  $x_n = y_m$  then  
    return  $1 + \text{LCS}(X^-, Y^-)$   
  else  
    return  $\max\{\text{LCS}(X, Y^-), \text{LCS}(X^-, Y)\}$ 
```

The algorithm makes 1 or 2 recursive calls and explores a tree of depth $O(n + m)$, therefore the time complexity is $2^{O(n+m)}$.

DP: tabulating

We need to find the correct traversal of the table holding the $c[i, j]$ values.

- Base case is $c[0, j] = 0$, for $0 \leq j \leq m$, and $c[i, 0] = 0$, for $0 \leq i \leq n$.
- To compute $c[i, j]$, we have to access

$c[i - 1, j - 1]$	$c[i - 1, j]$
$c[i, j - 1]$	$c[i, j]$

A row traversal provides a correct ordering.

- To being able to recover a solution we use a table b , to indicate which one of the three options provided the value $c[i, j]$.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Tabulating

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

LCS(X, Y)

for $i = 0$ to n do

$c[i, 0] = 0$

for $j = 1$ to m do

$c[0, j] = 0$

for $i = 1$ to n do

for $j = 1$ to m do

if $x_i = y_j$ then

$c[i, j] = c[i - 1, j - 1] + 1, b[i, j] = \nwarrow$

else if $c[i - 1, j] \geq c[i, j - 1]$ then

$c[i, j] = c[i - 1, j], b[i, j] = \leftarrow$

else

$c[i, j] = c[i, j - 1], b[i, j] = \uparrow.$

complexity:

$T = O(nm).$

Example.

$X=(ATCTGAT)$; $Y=(TGCATA)$. Therefore, $m = 6, n = 7$

		0	1	2	3	4	5	6
			T	G	C	A	T	A
0		0	0	0	0	0	0	0
1	A	0	↑0	↑0	↑0	↖1	←1	↖1
2	T	0	↖1	←1	←1	↑1	↖2	←2
3	C	0	↑1	↑1	↖2	←2	↑2	↑2
4	T	0	↖1	↑1	↑2	↑2	↖3	←3
5	G	0	↑1	↖2	↑2	↑2	↑3	↑3
6	A	0	↑1	↑2	↑2	↖3	↑3	↖4
7	T	0	↖1	↑2	↑2	↑3	↖4	↑4

Following the arrows: TCTA

Construct the solution

Access the tables c and d .

The first call to the algorithm is **sol-LCS**(n, m)

sol-LCS(i, j)

if $i = 0$ or $j = 0$ **then**

STOP.

else if $b[i, j] = \nwarrow$ **then**

sol-LCS($i - 1, j - 1$)

return x_i

else if $b[i, j] = \uparrow$ **then**

sol-LCS($i - 1, j$)

else

sol-LCS($i, j - 1$)

The algorithm has time complexity $O(n + m)$.

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Longest common substring

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- A slightly different problem with a similar solution
- **LCSt**: Given two strings $X = x_1 \dots x_n$ and $Y = y_1 \dots y_m$, compute their **longest common substring** Z , i.e., the largest k for which there are indices i and j with $x_i x_{i+1} \dots x_{i+k} = y_j y_{j+1} \dots y_{j+k}$.
- For example:
X : DEADBEEF
Y : EATBEEF
Z :

Longest common substring

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- A slightly different problem with a similar solution
- **LCS_t** Given two strings $X = x_1 \dots x_n$ and $Y = y_1 \dots y_m$, compute their longest common substring Z , i.e., corresponding to the largest k for which there are indices i and j with $x_i x_{i+1} \dots x_{i+k} = y_j y_{j+1} \dots y_{j+k}$.
- For example:
X : DEADBBEEF
Y : EATBEEF
Z : BEEF pick the longest substring

Characterization of optimal solution

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- Let $X = x_1 \cdots x_n$ and $Y = y_1 \cdots y_m$ and let Z be a longest common substring.
 - $Z = x_i \cdots x_{i+k} = y_j \cdots y_{j+k}$
 - Z is the longest common suffix of $X(i+k)$ and $Y(j+k)$.
- We can consider the subproblems $LCSf(i, j)$: compute the longest common suffix of $X(i)$ and $Y(j)$.
- The $LCSf(X, Y)$ is the longest of such common suffixes.

Computing the LC Suffixes

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

- To solve $LCSf(i, j)$ it is enough to go backward from position i in X and j in Y until we find two different characters.
- This has cost $O(n + m)$ per subproblem.
- We get a $O(nm(n + m))$ algorithm for LCSt
- **Can we do it faster?** Let us use DP!

A recursive solution for LC Suffixes

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Notation:

- $X[i] = x_1 \dots x_i$, for $0 \leq i \leq n$
- $Y[j] = y_1 \dots y_j$, for $0 \leq j \leq m$
- $s[i, j]$ = the length of the LC Suffix of $X[i]$ and $Y[j]$.
- Want $\max_{i,j} s[i, j]$ i.e., the length of the LCSt of X, Y .

DP approach: Recursion

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

Therefore, given X and Y

$$s[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 0 & \text{if } x_i \neq y_j \\ s[i - 1, j - 1] + 1 & \text{if } x_i = y_j \end{cases}$$

Using the recurrence the cost per recursive call (or per element in the table) is constant

Tabulating

DP technique

Guideline

W activity
selection

0-1 Knapsack

DP for pairing
sequences

Framework

Edit distance

Longest common
subsequence (LCS)

Longest common
substring

```
LCSf( $X, Y$ )  
for  $i = 0$  to  $n$  do  
     $s[i, 0] = 0$   
for  $j = 1$  to  $m$  do  
     $s[0, j] = 0$   
for  $i = 1$  to  $n$  do  
    for  $j = 1$  to  $m$  do  
         $s[i, j] = 0$   
        if  $x_i = y_j$  then  
             $s[i, j] = s[i - 1, j - 1] + 1$ 
```

complexity:

$O(nm)$.

Which gives an
algorithm with
cost $O(nm)$ for
LCSt